

1 B⁺-Tree Extensions

This section explores important extensions and variations of the B⁺-tree structure that improve efficiency and adapt it to practical database scenarios.

1.1 B⁺-Tree File Organization

Key Idea

Instead of traditional index-sequential files (which suffer from overflow blocks), a B⁺-tree can act both as an **index** and as a **file organizer**.

- Leaf nodes store the **actual records**, not just pointers.
- Non-leaf nodes maintain the index structure.

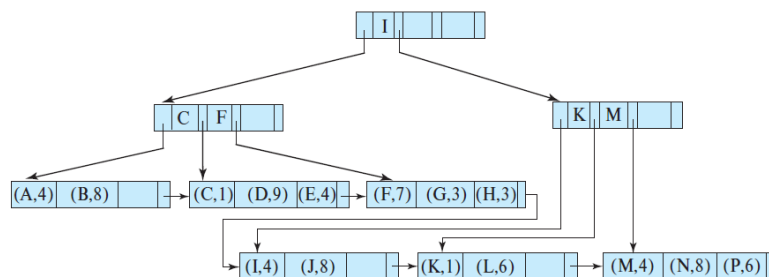


Figure 1: B⁺-tree file organization (records stored at the leaf level).

Insertion & Deletion

- Insert: Locate target block (largest key $\leq v$).
- If block full $\Rightarrow$ split and propagate upward.
- Delete: Remove record; fix underflow with redistribution or merging.

Space Utilization Improvement

Better Space Utilization

- Redistribute entries across **multiple siblings**.
- Guarantees:
 - After insertion split: at least $\lfloor \frac{2n}{3} \rfloor$ entries per node.
 - After deletion with 3-node redistribution: at least $\lfloor \frac{3n}{4} \rfloor$ entries.

1.2 Secondary Indices and Record Relocation

Problem

In B⁺-tree file organization, record **relocation** during splits breaks secondary index pointers.

Solution: Store **primary keys** in secondary indices (instead of direct pointers).

1. Use secondary index → retrieve primary keys.
2. Use primary index → locate actual records.

1.3 Indexing Strings

Challenges

- Strings are often **variable length** and **long**.
- This creates variable fanout (different number of entries per node).

Prefix Compression

- Store only distinguishing **prefixes** at internal nodes.
- Example: To separate “Silas” and “Silver”, store prefix “Silb”.

1.4 Bulk Loading of B⁺-Tree Indices

Why Bulk Loading?

Tuple-at-a-time insertion is expensive due to random I/O. Bulk loading sorts entries first, then builds the tree **bottom-up**.

1. Sort entries on search key.
2. Fill leaf nodes sequentially.
3. Build higher levels using minimum keys of child blocks.

1.5 B-Tree Index Files

- **Key difference:** Search keys appear only once.
- Requires **bucket pointers** in non-leaf nodes.

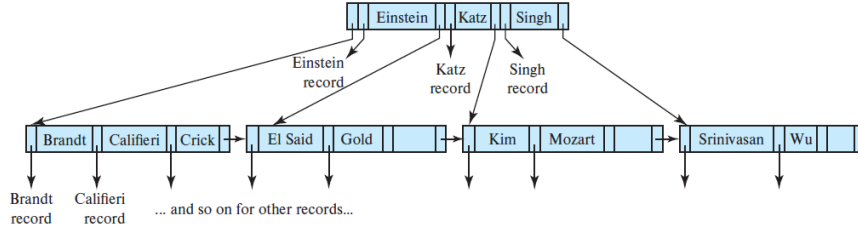


Figure 2: Comparison: B-tree vs B⁺-tree.

Pros vs Cons

Pros: Less redundant storage. **Cons:** Smaller fanout, deeper trees, more complex deletion.

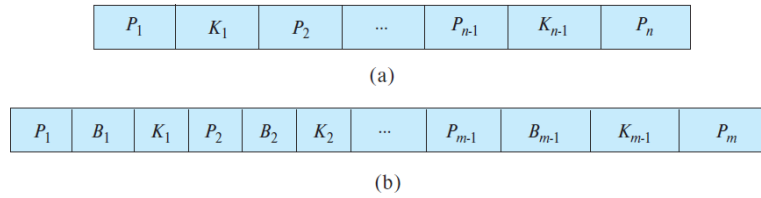


Figure 3: Generalized B-tree leaf and non-leaf nodes.

1.6 Indexing on Flash Storage

- SSDs: very fast random reads (20–100 μ s) vs. disk (5–10 ms).
- But writes are costly (erase-before-write).

Optimizations:

- Use smaller node size (match SSD page size).
- Buffer updates at higher nodes.
- Use **Log-Structured Merge Trees (LSM)** for batched updates.

1.7 Indexing in Main Memory

- B⁺-trees can be used directly for in-memory DBs.
- Optimizations:
 - Small node size = fewer cache misses.
 - Hybrid systems: large nodes structured as **cache-sized trees**.